Homework 7

(135 points)

Assigned: Thursday, April 2, 2026
Due: Friday, April 17, 2026

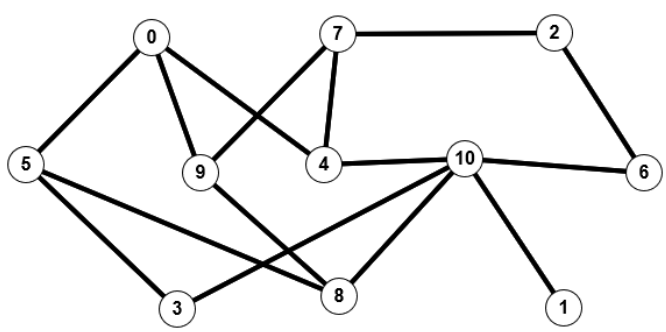
Purpose

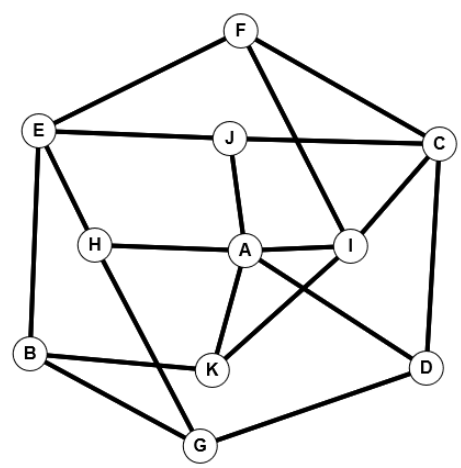
This assignment aims to reinforce core concepts in graph theory and to develop proficiency in graph-based algorithms.

Part A. Graph Traversal (20 pts)

Manually perform both depth-first search (DFS) and breadth-first search (BFS) on the following two graphs. For each graph, list the traversal order (sequence of visited nodes) starting from every node.

To ensure consistency, when a node has multiple unvisited neighbors, always select the next node in ascending order of node labels for both DFS and BFS. Complete this section in the online portion of Blackboard HW7. Due to the known issue in Blackboard, it is strongly recommended that you complete Part A well in advance and save your answers to prevent any last-minute submission delays.

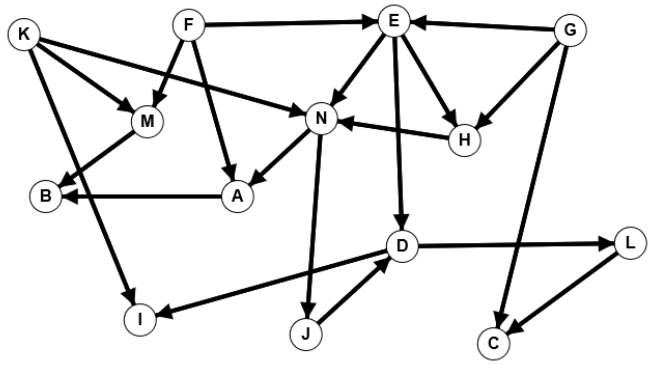




Part B. Toplogical Sort (10 pts)

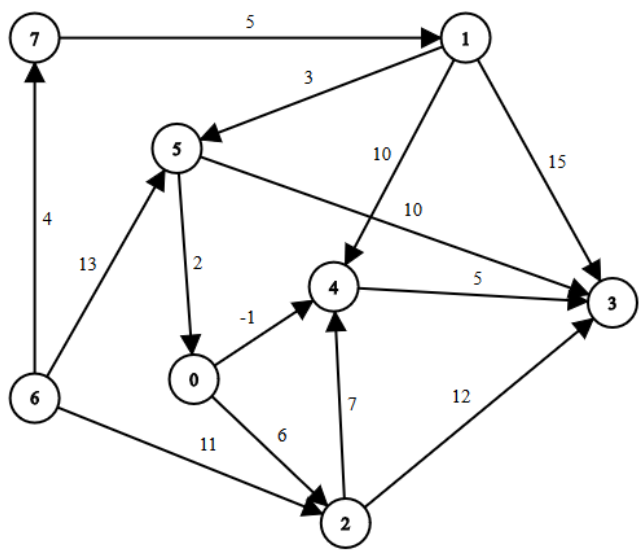
Sort the nodes in a valid topological order of the following directed acyclic graph (DAG) using the Source Removal Algorithm.

For consistency, when there is a choice, always select the node with the smallest label (in ascending order). Complete this section in the online portion of Blackboard HW7.



Part C. Single Source Shortest Path (SSSP) - DAG (25 pts)

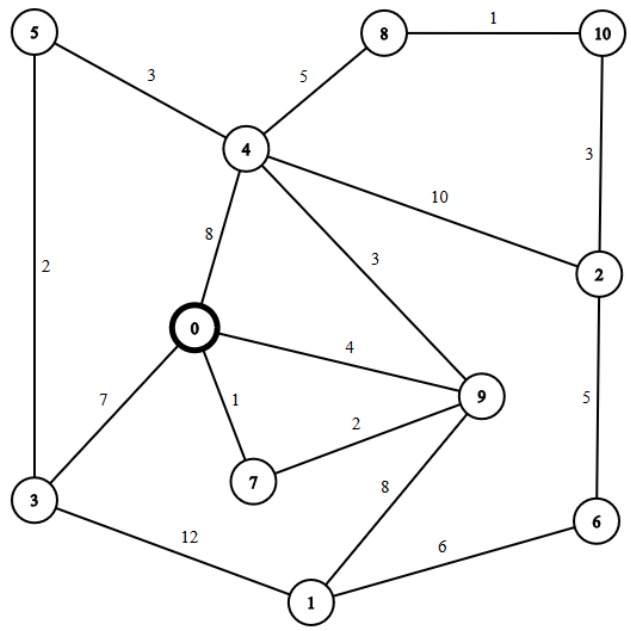
Perform the single-source shortest path (SSSP) algorithm on the following directed acyclic graph (DAG) using a DFS-based topological sorting approach. The edge weights represent the distances between nodes. The source node is 6.



1. Show the detailed steps for performing the DFS-based topological sort. Again for consistency, when there is a choice, always select the next node in ascending order of node labels. Provide the final topological ordering of the nodes.
2. Show the detailed steps for relaxing edges in topological order and updating the shortest path table. Present the final shortest path table, including distances and predecessor pointers (backtracking arrows) for all nodes.
3. Provide the shortest path from node 6 to node 3 by backtracking using the predecessor pointers. Write the path as a sequence of nodes (e.g., 1-2-3-4-5-6-7).

Part D. Single Source Shortest Path (SSSP) - General Graph (25 pts)

Perform the single-source shortest path (SSSP) algorithm on the given undirected graph using Dijkstra's algorithm. The edge weights represent the distance between nodes. The source node is 0.

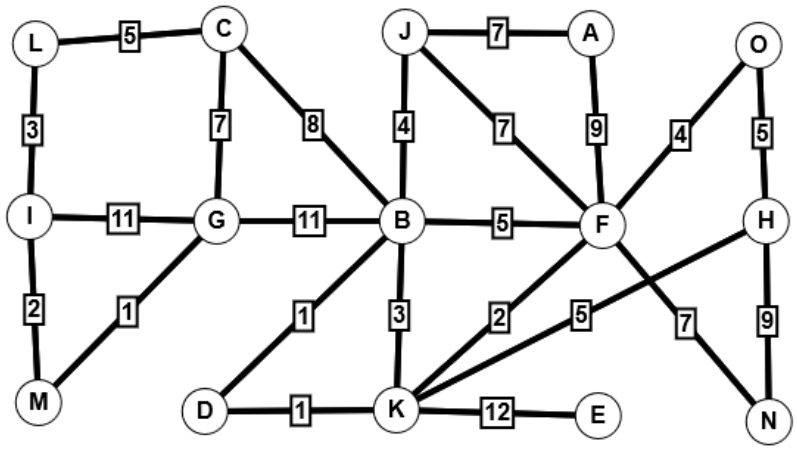


1. Show the detailed steps for relaxing edges and updating the shortest path table. Present the final shortest path table, ordered by node labels.

2. List the nodes in the order they are extracted (popped) from the priority queue (or min heap). Again for consistency, in the case of ties, the node with the smaller label should be extracted first.

Part E. Minimum Spanning Trees (MST) (10 pts)

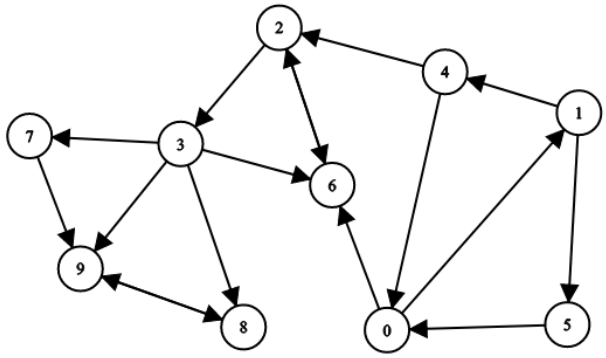
Find a minimum spanning tree (MST) for the given graph using Prim's Algorithm.



1. Show the step-by-step construction of the MST (one edge at a time), and clearly identify the final MST. Highlight the resulting tree and submit a clear scan or photograph of your work.
2. Compute the total cost of the MST.
3. Is there an alternative MST for this graph? Justify your answer.

Part F. Strongly Connected Components (SCC) (10 pts)

Identify all SCCs in the following directed graph.

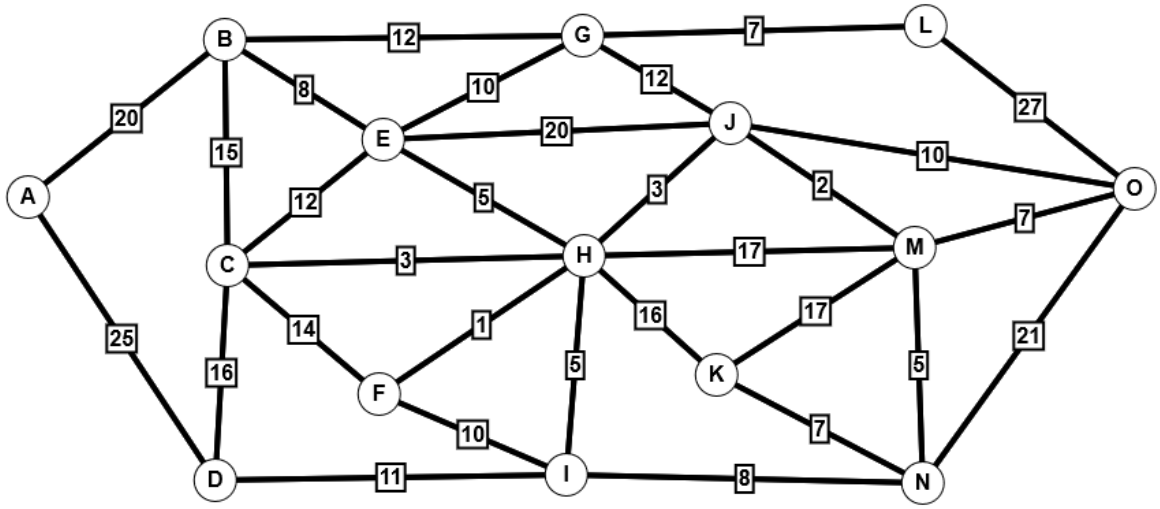


1. What is the total number of SCCs?
2. Show all SCCs.

Clearly indicate the SCCs directly on the graph (e.g., by circling each SCC) and take a clear photo or scan of your work.

Part G. Maximum Flow (10 pts)

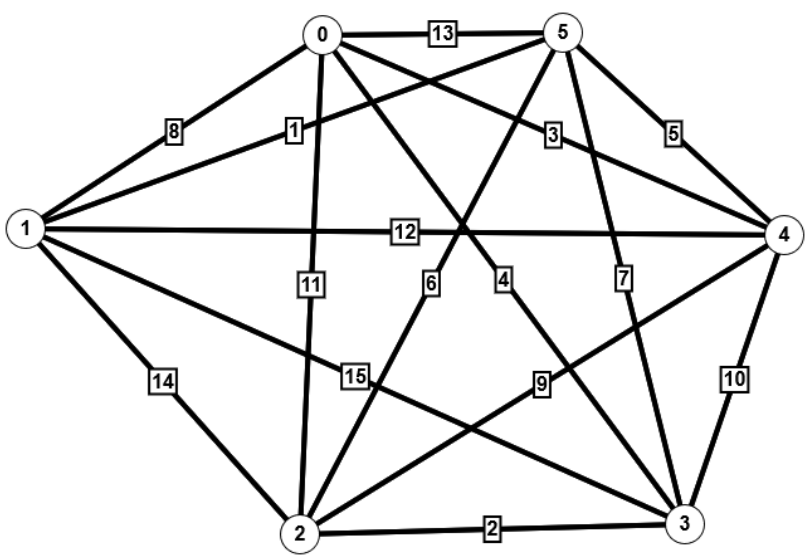
Find the maximum flow of the following graph using the maximum-flow minimum-cut theorem assuming node A is the source and node O is the sink.



1. Calculate the maximum flow.
2. Show the cut of edges.

Part H: Travelling Salesman Problem (TSP) (25 pts)

Solve the Traveling Salesman Problem (TSP) for the given graph by finding a Hamiltonian circuit with minimum total weight using the following two greedy (heuristic) algorithms:



1. Nearest-neighbor algorithm;
2. Multi-fragment algorithm.

For each algorithm:

1. Show the detailed steps, and clearly highlight the final Hamiltonian circuit on the original graph.

2. Compute the total cost (distance).

Additionally, answer the following questions:

1. Do both algorithms produce the same circuit?
2. Are the resulting circuits optimal? Justify your answer.
   Hint: To prove optimality, you must either exhaustively examine all possible circuits or provide a formal argument that no better circuit exists. To show non-optimality, it suffices to provide a counterexample, i.e., a circuit with a smaller total cost.

What to Submit

Submit all answers in a single `PDF` file via Blackboard before the due date. Additionally, complete and submit the online components for Part A and Part B on Blackboard.